Sim-to-Real with RL Fine-Tuning in Dynamic Air Hockey Tasks

Anonymous Author(s)
Affiliation
Address
email

Abstract: Dynamic tasks often require policies which are able to take precise, timed actions which are difficult to teleoperate, and for which demonstration data can struggle to sufficiently capture the task. Reinforcement learning offers a clear benefit here by self-optimizing good performance. However, these environments often have two major challenges: first, learning to perform a precise action from scratch, or even from a low-performing policy, will often result in poor performance or extremely high sample efficiency. Second, training RL often still requires substantial data before meningful improvements can be made. We address these issues by first training sim-to-real policies which can achieve passable performance on the robot, then utilize RL fine tuning to achieve impressive juggling, exceeding even the capabilities of a human player. We compare these methods against demonstration-based methods such as behavior-cloning and offline RL, as well as pure online RL learning, and show that our learning strategy acheives substantially improvement performance by comparison.

Keywords: CoRL, Robots, Reinforcement Learning, Sim-to-Real

1 Introduction

Submission to CoRL 2026 will be entirely electronic, via a web site (not email). Information about the submission process and LaTeX templates are available on the conference web site at https://corl.org/. For camera ready submission, use the `final` option for the `\usepackage` command.

2 Method

Our approach has three stages: (i) a *loose* system identification of the Box2D simulator to roughly match the real robot, (ii) policy training in simulation with domain randomization to produce a policy that transfers, and (iii) online fine-tuning on the real robot using residual RL on top of the transferred policy.

2.1 Loose System Identification

We perform two coarse system-identification steps that fit the simulator's free-puck dynamics and paddle controller to real-world teleoperation recordings. "Loose" is deliberate: the goal is approximate fidelity such that domain randomization can absorb the residual gap, not exact trajectory replay.

Puck dynamics. The free-flight puck is modeled with linear damping and a tilt-induced constant acceleration along the table's long axis,

$$\frac{d\mathbf{v}}{dt} = -\gamma\mathbf{v} - \mathbf{g}, \qquad \mathbf{g} = (g_x, 0), \tag{1}$$

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

which admits a closed-form solution for $\mathbf{x}(t)$ given $(\mathbf{x}_0, \mathbf{v}_0)$. We fit the two scalars (g_x, γ) by minimizing mean per-frame position error on real puck-only trajectory segments collected from teleoperation. The chosen values are $g_x = -0.661$ and $\gamma = 0.178$. In practice g_x is well-identified, while γ is weakly identified (a wide range of damping values fit comparably well).

Paddle controller. The simulated paddle is driven by a PID controller that produces a force on the paddle body from the commanded position error:

$$\mathbf{F} = K_p(\mathbf{p}_{\text{des}} - \mathbf{p}) + K_d \frac{d}{dt}(\mathbf{p}_{\text{des}} - \mathbf{p}) + K_i \int (\mathbf{p}_{\text{des}} - \mathbf{p})\, dt. \tag{2}$$

The paddle's effective response depends jointly on the PID gains and on the simulated paddle inertia, which we control through a Box2D `paddle_density` knob. We fit $(K_p, K_d, K_i, \texttt{paddle\_density})$ by grid search, scoring each configuration by paddle position error against teleop trajectories replayed into the sim. The chosen values are $K_p = 9000$, $K_d = 50$, $K_i = 0$, $\texttt{paddle\_density} = 3000$. Higher K_p and lower K_d than the legacy defaults consistently improve tracking, and the fitted density matches real paddle inertia substantially better than the legacy value.

Wall and contact parameters (paddle–puck and wall restitution, friction) are only sanity-checked. Paddle–puck collisions in particular are left coarse: we expect the residual fine-tuning stage (Section 2.3) to absorb the remaining contact-modeling gap.

2.2 Sim Training with Domain Randomization

We train in simulation under randomization across several axes to produce a policy robust enough for sim-to-real transfer.

Starting distribution. Each episode begins with the puck either near the paddle spawn (15%) or at a randomized location near the top of the table (85%).

Observation noise and occlusion. Puck position observations are perturbed with noise. We additionally inject occlusion events that drop the position observation, with a higher occlusion rate when the puck is close to the paddle (mirroring real-world tracking failures in that region).

Observation delay. The observation pipeline applies a delay between the true state and what the policy sees, with the delay magnitude itself randomized across episodes.

Action randomization. With 30% probability per step, the commanded action is rescaled to between 25% and 75% of its original magnitude.

Collision randomization. Wall bounces are perturbed by a randomized angle within a 10-degree cone. Paddle–puck collisions are perturbed by both a randomized angle within a 10-degree cone and a randomized strength.

2.3 Online Fine-Tuning with Residual RL

After deploying the sim-trained policy on the real robot, we fine-tune online using residual RL: a learned residual head adds a correction on top of the frozen base policy's action. The two primary axes of variation we study are (i) the *residual action scale*, which bounds how much the residual head can deviate from the base policy, and (ii) the *data balance* in the replay buffer between newly collected real-world transitions and prior experience.

3 Citations

Citations can be made using either \citep{} or \citet{}, depending from the appropriateness. To avoid the citation moving to the next line, it is often a good practice to replace the space before with a tilde (~) character. Example 1: "CoRL is the best conference ever [1]." Example 2: "Lagrange [2] proved, both theoretically and numerically, that CoRL is the best conference ever."

4 Experimental Results

4.1 Evaluation Protocol

We evaluate policies on two metrics, both reported over sliding windows of recent episodes:

- **Task success rate.** An episode is counted as a success if the policy juggles the puck at least three times. We report the sliding-window success rate.
- **Task reward.** The episodic reward, summarized over the same window via mean, standard deviation, minimum, and maximum.

Both metrics are reported across sliding windows of size 5, 10, 25, and 50 episodes to capture short-term behavior as well as longer-term stability.

5 Ablations

We ablate the components of our pipeline along two axes: (i) what matters for sim-to-real *zero-shot* transfer (i.e., the simulation training pipeline that produces the base policy) and (ii) what matters for online *fine-tuning* on top of that policy. We emphasize that the specific engineering choice underlying each component is not the contribution: each row below admits many reasonable alternative implementations, and our intent is to surface which categories of intervention are load-bearing rather than to claim our particular instantiations are uniquely correct.

We develop and verify pipeline-level interventions in simulation wherever possible, since real-robot experiments are substantially more expensive in wall-clock time, hardware wear, and human supervision. Zero-shot ablations are evaluated only in the sim-to-real direction — a sim-to-sim version is not informative here, since the sim-trained policy is by construction the optimal policy for the sim it was trained in. Online fine-tuning ablations are evaluated in both sim-to-sim and sim-to-real settings. The sim-to-sim setting is fabricated: the evaluation simulator differs from the training simulator in numerous small ways, with the dominant difference being a **35% reduction in paddle radius** (from $0.0508\,\text{m}$ to $\sim 0.0330\,\text{m}$), which approximates contact-tracking degradations we observe on the real robot.

5.1 Zero-Shot Sim-to-Real Ablations

System identification. Our default sim parameters come from a coarse fit to real-world data — gravity ($g_x = -0.661$), puck damping (0.178), paddle density (3000), and PID gains (K_p=9000, K_d=50, K_i=0); see §2.1. The ablation replaces these with off-the-shelf Box2D defaults (effectively undoing the sysid), holding all other randomization on.

Collision randomization. Two independently toggleable mechanisms perturb collision outcomes per event: wall direction is rotated within a $\pm 10°$ cone, and paddle–puck collisions are rotated within a $\pm 10°$ cone with an impulse-strength multiplier drawn from $U[0.5, 1.0]$. We ablate by disabling each.

Action randomization. At each control step, with probability 0.30, the commanded force is rescaled by a factor drawn uniformly from $[0.25, 0.75]$, modeling unmodeled actuator-side losses. We ablate by disabling.

Starting data distribution. Episodes start with the puck near the paddle spawn with probability 0.15 and at a randomized location near the top of the table otherwise. We ablate against 100% near-top and 100% near-paddle starts.

Real-world starting states. As a complement to the synthetic mixture above, we ablate initializing the simulator's paddle and puck positions (and velocities, where available) from the empirical distribution of starting states observed on the real robot — drawn from prior real-world rollouts and teleoperation traces — rather than from the hand-designed mixture. The hypothesis is that closing the starting-state distribution gap reduces the share of the sim-to-real gap that domain randomization needs to absorb.

Observation randomization. Three components: (a) additive Gaussian noise on puck position with std $0.01\,\mathrm{m}$; (b) random occlusions that drop the puck observation, with a base rate of $\sim 2.5\%$ amplified $3\times$ within $5\,\mathrm{cm}$ of the paddle; and (c) observation delay of $25\,\mathrm{ms}$ randomized by $\pm 25\%$ per episode. We ablate by disabling each component.

5.2 Online Fine-Tuning Ablations

Exploration. Our standard residual fine-tuning uses TD3-style Gaussian action noise with std 0.05 (lower than the 0.1 used during from-scratch sim training, since the base policy already explores meaningfully). We ablate the noise scale across $\{0, 0.05, 0.1, 0.2\}$ and additionally toggle the structured exploration primitives (e.g., directional pushes, target-position drives) that are off by default in residual fine-tuning.

Residual scale. The residual head's output is bounded to a fraction of the action range; the standard value is 0.15, sweeping $\{0.05, 0.15, 0.30\}$. Smaller values keep the policy close to the transferred base; larger values give the residual more authority but increase drift risk.

Replay buffer staleness. Two coupled knobs control how the buffer prioritizes recent vs. old data: an exponential age decay applied to PER priorities (`priority_age_decay`, standard 10^{-4}), and a recency-weighted success/failure split (`success_top_fraction`, standard 0.15 for the big-gap recipe). We ablate against `priority_age_decay` $= 0$ (no decay) and sweep `success_top_fraction` $\in \{0.15, 0.30, 0.50\}$.

Q overestimation. Standard TD3 uses twin critics ($N{=}2$) with a min-of-two target. We sweep min-of-N ensembles for $N \in \{2, 3, 5\}$, and additionally evaluate REDQ-style target estimation (random subset min) at $N{=}5$ and $N{=}10$, motivated by an observed Q-overestimation pathology during residual fine-tuning where Q1 grows ∼2.6–4× over training.

6 Conclusion

Table 1: Description of ablations. Each row names a component, the standard value used in our pipeline, and the variants we sweep. Values are quoted from the configs at `scripts/smooth_policy/amp_history/configs/`.

Component	**Standard setting**	**Ablation**
Zero-shot (sim-to-real) — training-pipeline ablations		
System identification	Fitted: $g_x = -0.661$, puck damping 0.178, paddle density 3000, PID $(9000, 50, 0)$	Off-the-shelf Box2D defaults
Collision randomization	Wall direction $\pm 10^\circ$; paddle–puck direction $\pm 10^\circ$ + strength $\sim U[0.5, 1.0]$	Disable each independently
Action randomization	Per-step p=0.30, force rescaled by $\sim U[0.25, 0.75]$	Disable
Starting distribution	15% near paddle / 85% near top of table	100% top; 100% near paddle
Real-world starting states	Synthetic mixture above (no real-world init)	Initialize paddle/puck from empirical distribution of real-world rollouts
Observation randomization	Puck-position noise std 0.01 m; occlusion base 2.5% ($3\times$ near paddle); delay 25 ms $\pm$ 25%	Disable each component
Online fine-tuning — evaluated sim-to-sim (35% smaller paddle) and sim-to-real		
Exploration	TD3 Gaussian noise std 0.05; primitives off	Sweep std $\in \{0, 0.05, 0.1, 0.2\}$; toggle primitives
Residual scale	0.15	$\{0.05, 0.15, 0.30\}$
Replay buffer staleness	`priority_age_decay` $= 10^{-4}$; `success_top_fraction` $= 0.15$	Disable age decay; sweep top-fraction $\in \{0.15, 0.30, 0.50\}$
Q overestimation	Min-of-2 critics (twin TD3)	Min-of-N for $N \in \{3, 5\}$; REDQ random-subset min at $N \in \{5, 10\}$

References

[1] C. F. Gauss. Theoria motus corporum coelestium in sectionibus conicis solem ambientium. *Placeholder Journal*, 1857.

[2] J.-L. Lagrange. Mécanique analytique. *Placeholder Journal*, 1788.